

## Experiment-11

**Name:** - C. Jyothi

**Regno:** -192425312

**Course Code:** -MLA0305

**Slot:** B

---

### Experiment 11 – A3C

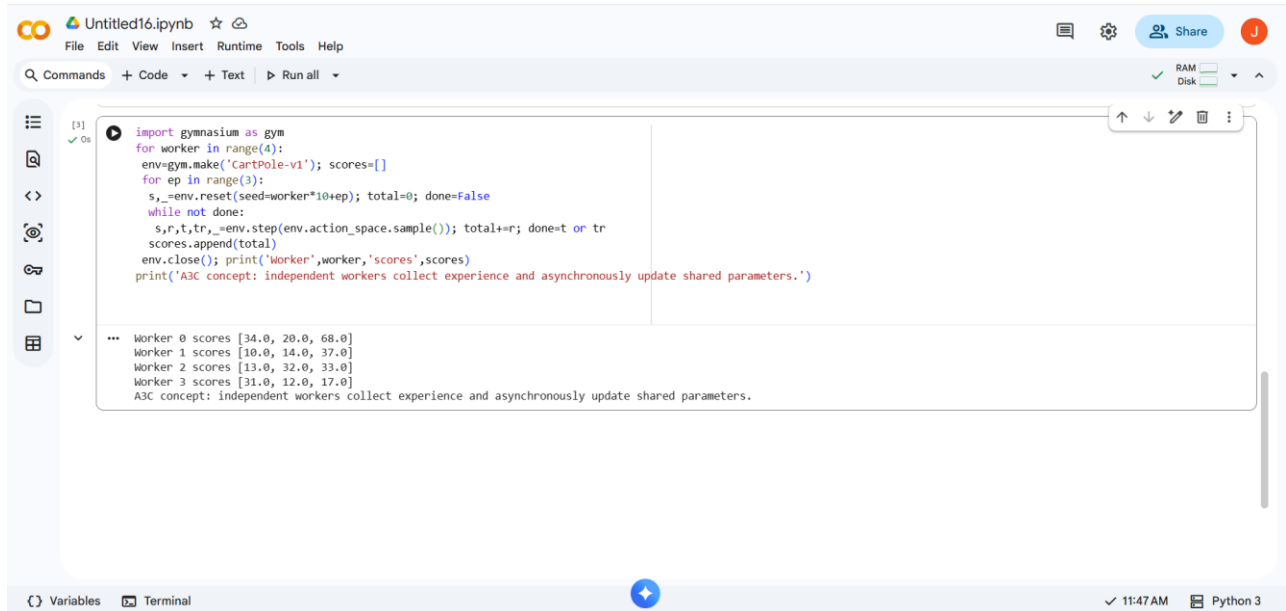
**Title:** Implementation and Performance Comparison of A2C and A3C

**Aim:**

To implement the Asynchronous Advantage Actor-Critic algorithm and compare its convergence performance with A2C.

**Procedure:**

1. Create a suitable Reinforcement Learning environment.
2. Define shared Actor-Critic network parameters.
3. Create multiple worker agents.
4. Allow each worker to interact with its own environment instance.
5. Calculate advantages from worker experiences.
6. Compute local gradients.
7. Update the shared global network asynchronously.
8. Train the A3C model for multiple episodes.
9. Train an A2C model under comparable conditions.
10. Compare reward curves and convergence speed.



The screenshot shows a Jupyter Notebook window titled 'Untitled16.ipynb'. The code cell contains the following Python code:

```
[1] import gymnasium as gym
for worker in range(4):
    env=gym.make('CartPole-v1'); scores=[]
    for ep in range(3):
        s,_=env.reset(seed=worker*10+ep); total=0; done=False
        while not done:
            s,r,t,tr,_=env.step(env.action_space.sample()); total+=r; done=t or tr
            scores.append(total)
        env.close(); print('Worker',worker,'scores',scores)
print('A3C concept: independent workers collect experience and asynchronously update shared parameters.')
```

The output of the code cell is:

```
... Worker 0 scores [34.0, 20.0, 68.0]
Worker 1 scores [10.0, 14.0, 37.0]
Worker 2 scores [13.0, 32.0, 33.0]
Worker 3 scores [31.0, 12.0, 17.0]
A3C concept: independent workers collect experience and asynchronously update shared parameters.
```

## Result:

A3C was successfully implemented using multiple workers. Its convergence behavior was compared with A2C, demonstrating the effect of asynchronous parallel learning.